

Tercer proyecto

El objetivo de este tercer proyecto es simular un sistema de archivos remoto basado en buckets al estilo AWS S3. Para eso cada bucket será simulado con un único archivo en el servidor. Se deberá crear un programa servidor llamado *aws-s3_server* que implemente diferentes funciones similares a AWS S3, junto con otro programa cliente llamado *aws-s3* que se comuniquen con dicho servidor mediante sockets y permita gestionar archivos locales y remotos.

AWS S3

AWS S3 (Simple Storage Service) es un servicio de almacenamiento de objetos en la nube, no un sistema de archivos tradicional. A diferencia de un disco duro o un sistema de archivos como NTFS o ext4, S3 no organiza los datos en una estructura jerárquica real de directorios y subdirectorios. En su lugar, utiliza una arquitectura completamente plana donde cada archivo (llamado "objeto") se guarda en un espacio de nombres único y se identifica mediante una "clave" (key). Para acceder a estos objetos, se usa una API REST a través de HTTP (operaciones como GET, PUT, DELETE), y los objetos son inmutables, lo que significa que no se puede modificar parte de un archivo; para cambiarlo hay que reemplazarlo entero.

Un **Bucket** es el contenedor de más alto nivel en S3, equivalente a una unidad de almacenamiento raíz. Cada bucket tiene un nombre único a nivel global en toda AWS, y dentro de él se almacenan los objetos. Los buckets son el elemento sobre el cual se configuran aspectos como los permisos de acceso, la región donde se almacenarán los datos, el registro de accesos (logs), y las políticas de ciclo de vida (por ejemplo, mover objetos a almacenamiento más económico después de cierto tiempo). No se pueden anidar buckets unos dentro de otros, y para eliminar un bucket, primero debe estar vacío (a menos que se fuerce su eliminación).

Las **carpetas simuladas**, también llamadas **prefijos**, son una ilusión visual para parecerse a los directorios tradicionales. En realidad, lo que S3 guarda es la clave completa del objeto incluyendo las barras diagonales. Por ejemplo, si se sube un archivo con la clave "fotos/vacaciones/2024/playa.jpg", no existen carpetas llamadas "fotos", "vacaciones" o "2024". Todo es un único string que contiene slashes, y las herramientas de AWS (como la consola web o el comando `aws s3 ls`) interpretan esos slashes para mostrar una estructura similar a carpetas. Esto permite subir directamente un archivo a una ruta "profunda" sin necesidad de crear previamente ninguna carpeta intermedia.

Cliente AWS-S3

Los siguientes comandos podrán ser ejecutados mediante el programa cliente **aws-s3** y deberán ser enviados al servidor para su ejecución:

Comando	Descripción	Ejemplo de Uso
aws-s3 ls	Lista buckets o el contenido de un bucket y sus prefijos (carpetas simuladas) .	aws-s3 ls s3://mi-bucket/mi-carpeta/
aws-s3 mb	Crea un nuevo bucket.	aws-s3 mb s3://mi-nuevo-bucket
aws-s3 cp	Copia archivos entre un directorio local y S3, o entre dos ubicaciones en S3 .	aws-s3 cp archivo.txt s3://mi-bucket/ aws-s3 cp s3://mi-bucket/archivo.txt ./
aws-s3 mv	Mueve un archivo u objeto, lo que equivale a una copia seguida de una eliminación del origen .	aws-s3 mv archivo.txt s3://mi-bucket/
aws-s3 rm	Elimina un objeto o varios objetos de un bucket. Para eliminar una carpeta entera, usa la opción --recursive .	aws-s3 rm s3://mi-bucket/archivo.txt aws-s3 rm s3://mi-bucket/mi-carpeta/ --recursive
aws-s3 sync	Sincroniza el contenido de un directorio local con un bucket de S3, o viceversa. Es muy útil para respaldos o despliegues, ya que solo copia los archivos nuevos o modificados .	aws-s3 sync ./mi-carpeta-local s3://mi-bucket/mi-carpeta/ --delete
aws-s3 rb	Elimina un bucket. Por defecto, solo puede eliminar buckets vacíos. Se puede forzar la eliminación de un bucket con contenido usando --force .	aws-s3 rb s3://mi-bucket aws-s3 rb s3://mi-bucket --force

Opciones Útiles para los Comandos

Los comandos anteriores se pueden combinar con opciones para modificar su comportamiento:

- **--recursive**: Obligatorio para ejecutar un comando (**cp**, **mv**, **rm**, **sync**) sobre todos los archivos de un directorio o prefijo .
- **--delete**: Utilizado con **sync**, elimina en el destino los archivos que ya no existen en el origen, manteniendo una réplica exacta .

Servidor AWS-S3

El programa servidor debe recibir y procesar los comandos enviados por el cliente. Aunque es posible utilizar un protocolo HTTP (con métodos como GET, PUT y DELETE), su uso no es obligatorio; también se puede implementar un protocolo de texto propio sobre sockets. Pero tome en cuenta con los archivos que se envían y reciben pueden ser binarios (imágenes, programas, etc), por lo que el protocolo debe permitir su correcta transmisión.

Para la gestión de los buckets, se utilizará un único archivo que contendrá todos los datos de los distintos archivos copiados al bucket específico. Al comienzo de este archivo se dispondrá de un bloque de directorio que liste los "prefijos" (paths) de cada archivo almacenado, junto con la posición (número de byte) dentro del archivo donde comienza su contenido. A continuación, el contenido de cada archivo se copiará de forma secuencial. Note que se pueden crear y consultar múltiples archivos de bucket (de hecho los comandos pueden copiar y mover archivos entre buckets).

Si se sube un archivo que ya existía se debe verificar primero si la cantidad de bytes del archivo es igual a la versión anterior, de no ser así se deberá copiar al final del archivo y marcar como libre el espacio que ocupaba anteriormente el archivo. Con este fin, se debe mantener una lista de espacios libres (dentro del bloque de directorios) de la cual se podrá echar mano cuando se requiera asignar un archivo nuevo. Usted puede elegir que algoritmo de asignación utilizará para administrar esta lista de espacios libres (primer ajuste, peor ajuste, mejor ajuste, mejor ajuste). Esta lista también se utilizará cuando se borra un archivo.

Análisis de resultados

Se deberán realizar una serie de pruebas usando diferentes estructuras de directorios y tipos de archivos, en donde se muestre la correcta ejecución de los comandos y recuperación de los archivos.

Toda la secuencia de pruebas deberá incorporarse al documento final del proyecto. Puede hacer copia de la salida de la consola (de los comandos usados) para mostrar los archivos copiados o recuperados al directorio y el correcto contenido de los archivos recuperados.

Consideraciones generales

- Todo el desarrollo del proyecto debe realizarse en lenguaje C y sobre ambiente Unix.
- No se permite la utilización de librerías externas diferentes a las proporcionadas por el lenguaje C.
- Se deberá generar una documentación formal, en formato pdf, en donde se describan las diferentes etapas del desarrollo del proyecto, las decisiones de diseño que se tomaron, los mecanismos de programación utilizados, y los resultados de las diferentes pruebas al programa. Dicha documentación deberá incluir al menos las siguientes secciones:
 - Introducción
 - Descripción del problema (este enunciado)
 - Definición de estructuras de datos utilizadas
 - Descripción detallada y explicación de los componentes principales del programa
 - Mecanismo de creación de archivos y comunicación con el servidor
 - Pruebas de rendimiento
 - Conclusiones
- El proyecto puede realizarse en grupos de dos estudiantes, bajo ninguna situación se permitirán grupos de más de dos personas.
- No se permite la copia de código entre grupos de estudiantes, o entregar código realizado por estudiantes los semestres anteriores, tampoco es permitido utilizar librerías adicionales (desarrolladas por terceros), o el uso de motores de IA para resolver este proyecto.